

Modern Application Development II Project Report

1. Student Details:

Name: Rituparna Bhaumik

Roll Number: 24F1002607

Email: 24f1002607@ds.study.iitm.ac.in

About me: I am a Chartered Accountant and currently a diploma level student at IIT Madras.

2. Project Details:

Project Title: Hospital Management System – HealixCare

Project Statement: To design and build a multi user web application that allows hospitals to manage patient appointments and treatment history and and patients to search for doctors and book appointments.

Approach: The app was built using Flask as backend and other technologies listed in this document. This was created keeping in mind to make it user friendly i.e. users can interpret the interface with ease. The admin is the superuser and controls the functionality of the app in terms of adding doctors and viewing the patient details (treatment history etc). The doctors can provide their week's availability by logging in to their dashboards and update patient treatment records. Patients can also login and book appointments. They can also view and download their treatment history. The app also has built in asynchronous tasks like sending daily reminders to the patients on their upcoming appointments and monthly report to doctors.

3. AI/LLM Declaration:

I used ChatGPT to assist me in assisting me with some API documentation and formatting. The extent of AI/LLM usage is around 12%, limited to code suggestions and UI formatting.

All final implementation logic, debugging, and integration were done manually.

4. Technologies and Framework Used:

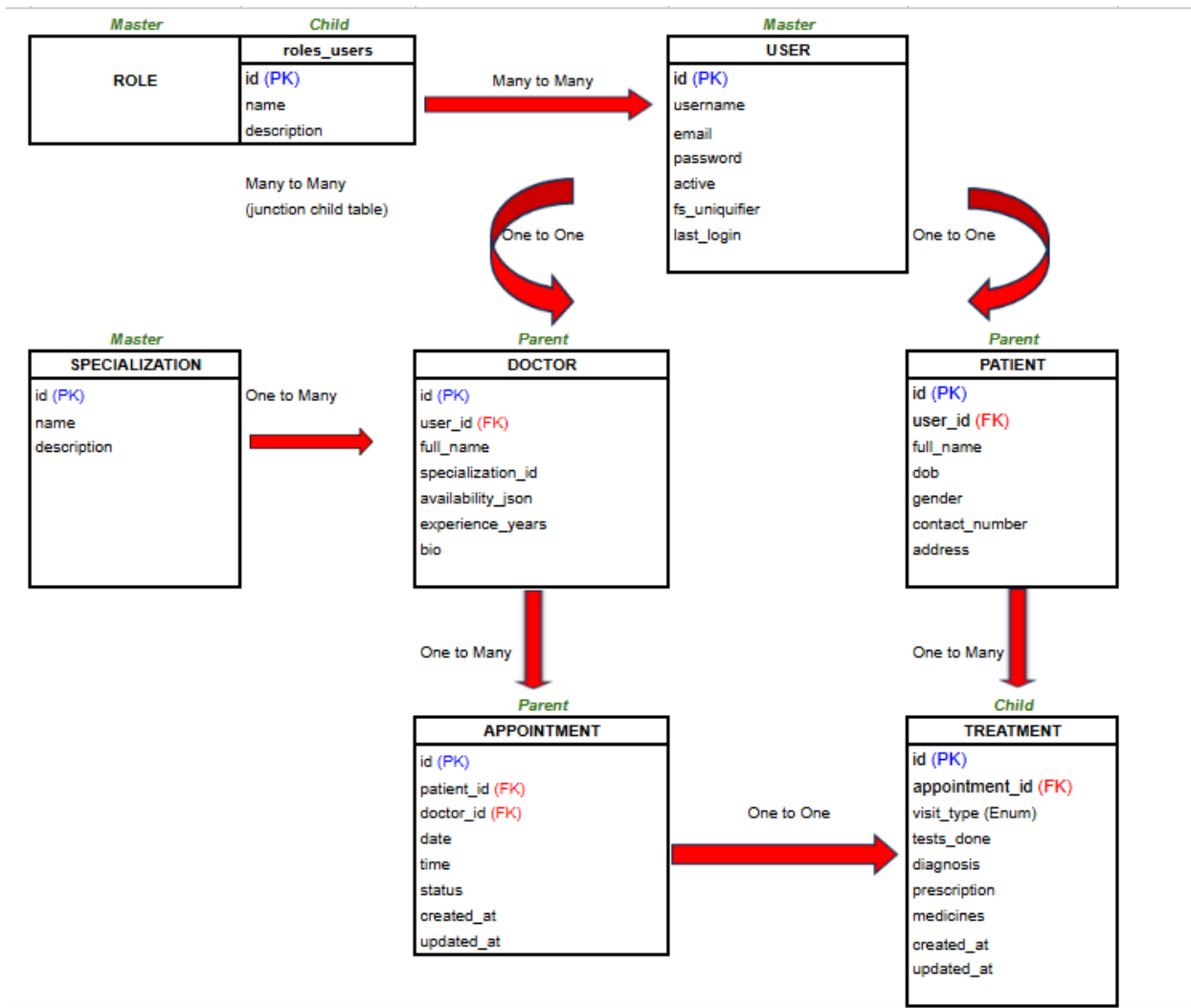
- Flask: Core backend web framework
- Flask-login: User authentication and session management
- SQLAlchemy: Object Relational Mapper for SQLite database
- Vue CDN: For frontend
- Bootstrap 5: Frontend styling
- SQLite: Local database
- Redis: Caching
- Redis and Celery: For batch jobs

5. Database Scheme/ ER Diagram:

Below is the ER diagram for the web app which shows the parent-child relationship between the tables. There are 7 tables: 1) Roles, 2) Users 3) Specialization 4) Doctor, 5) Patient, 6) Appointment, 7) Treatment. The role class defines the roles that users can take which is admin, doctor and patient. Users table stores the email, username, password and role of a user. Specialization stores the departments present in a hospital. In this app we have pre-defined 7 departments. The Doctor table stores the details of the doctor such as name, specialization, timing availability, experience in years. The Patient table stores the name, contact and appointment details. The Appointment table allows to store the timings for doctor bookings and the status of these appointments can be: booked, cancelled,

Modern Application Development II Project Report

completed. Different types of appointments have also been defined in the model. The Treatment table stores the visit type, tests done, diagnosis, prescription, and medicines.



6. API Resource Endpoints:

The API's have been broadly categorized into 3:

1. Admin related APIs:

GET /api/admin/admin_dashboard - View all doctors, patients, upcoming & past appointments

GET /api/admin/search - Search for doctors, patients, departments

POST /api/admin/new_doctor - Add a new doctor

GET /api/admin/doctor/<id> - Get doctor details

PUT /api/admin/doctor/<id> - Update doctor info

DELETE /api/admin/doctor/<id> - Delete doctor

POST /api/admin/doctor/<id>/block - Block doctor account

POST /api/admin/doctor/<id>/unblock - Unblock doctor account

GET /api/admin/patient/<id> - Get patient details

PUT /api/admin/patient/<id> - Update patient info

DELETE /api/admin/patient/<id> - Delete patient

Modern Application Development II Project Report

POST /api/admin/patient/<id>/block - Block patient account
POST /api/admin/patient/<id>/unblock - Unblock patient account
GET /api/admin/patient/<id>/appointments - View all patient appointments
GET /api/admin/patient/<id>/doctor/<id>/appointments - View specific doctor-patient visit history
GET /api/specializations - List all specializations

2. Doctor related APIs:

GET /api/doctor/me - Get current doctor profile & availability
GET /api/doctor/appointments - View all assigned appointments
GET /api/doctor/patients - Get distinct patients with appointments
GET /api/doctor/patient/<id> - Get specific patient details
GET /api/doctor/patient/<id>/appointments - Get patient appointment history with treatment details
POST /api/doctor/appointment/<id>/treatment - Add/update treatment for appointment
POST /api/doctor/appointment/<id>/complete - Mark appointment as complete
POST /api/doctor/appointment/<id>/cancel - Cancel appointment
PUT /api/doctor/availability - Update doctor's availability schedule
POST /api/logout - Logout

3. Patient related APIs:

GET /api/patient/me - Get logged-in patient profile
GET /api/patient/profile - Get patient profile
PUT /api/patient/profile - Update patient profile
GET /api/patient/patient_dashboard - View appointments overview
GET /api/patient/departments - List all departments/specializations
GET /api/patient/departments/<id>/doctors - Get doctors in department
GET /api/patient/search_doctors - Search doctors by specialization & date
GET /api/patient/doctor_availability/<id> - Get doctor availability with booked slots
POST /api/patient/book_appointment - Book appointment with doctor
GET /api/patient/appointments/history - Get past appointments
GET /api/patient/history - Full appointment history with treatment details
POST /api/appointments/<id>/cancel - Cancel appointment
POST /api/logout - Logout

Other backend related APIs have also been created to run the asynchronous tasks.

7. Architecture & Features:

Architecture: The hospital management app root folder has the following folders:

- 1) **app.py:** This is the main python page where the app is running
- 2) **/application:** This consists of routes (admin, doctor and patient), models.py, celery_init.py, backend task routes
- 3) **/static:** This includes 3 sub-folders: a) components: all .js components for admin, patient and doctor b) Images and c) styles: css base file. This also includes the script.js – the main js component.
- 4) **/templates:** Includes the index.html and the monthly mail reminder
- 5) **celery_config.py:** This file contains the configuration related to celery and the broker details i.e. redis

Features Implemented: All core functionalities have been implemented including all backend jobs. It also includes the following:

- 1) Prevent multiple appointments at the same date and time for the same doctor.

Modern Application Development II Project Report

- 2) Update appointment status dynamically (**Booked → Completed → Cancelled**).
 - 3) Admin and Patient should be able to search for a specialization or by a doctor's name
 - 4) Admin should be able to search patients by name, ID, or contact information.
 - 5) Store all completed appointment records for each patient.
 - 6) Include diagnosis, medication, and prescriptions for each visit.
 - 7) Allow patients to view their own treatment history.
8. **Video link:**
https://drive.google.com/file/d/11Zx4IqRdG-t9X1AkO_z6roYeMgatVDv6/view?usp=drive_link